

The CTX-anti family: two ways context-sensitive coverage backfires

Backdrop. *CTX-anti* (CTX-A) collects the roadblocks where *context-sensitive coverage hurts*: the plain coverage arm (*naive*) resolves the branch, but the same fuzzer with the call-context hook (*naive_ctx*) blocks it. *CtxHook* folds a hash of the live call-context (the caller chain) into every edge-map index, so the *same* control-flow edge counts as *new* coverage under each distinct caller. That extra dimension is exactly the I2S-anti story turned to feedback: the mechanism meant to refine guidance instead *inflates the coverage map*, and the campaign spends its budget chasing context-redundant novelty instead of the bytes the gate wants. The decisive pair is always the same — *naive* $\succ$ *naive_ctx* — because adding only the context dimension is the single-technique delta. Counts are validated roadblocks (n); the family has $n = 4$ validated in *one* category with two measurement routes, plus a residual shallow-literal tail (§2) that is honestly *inconclusive*.

A note on the signal. Context-sensitive coverage is a *non-local feedback* (like value-profile): it leaves no event in any single seed’s mutation lineage. The only handles are what the two corpora *reach* or *hoard*. The one validated category has *two measurement routes* for the same backfire, depending on which handle fires: corpus *size* (the ctx arm hoards a bloated corpus, route A) or per-seed *depth* (the flat arm drives the gate deeper while the ctx arm fragments and stalls, route B). Figure 1 shows both side by side.

1. The context dimension inflates coverage redundantly *ctx_inflation* ($n = 4$)

Context-sensitive coverage makes the *same* control flow look novel under each call-context, so *naive_ctx* spends the campaign chasing context-redundant “new” coverage instead of the bytes the gate wants. This backfire surfaces through **two measurement routes** — the ctx arm hoards a bloated *corpus* (route A), or it fragments a deep path so no seed reaches the gate *depth* the branch needs (route B). Same anti-mechanism (redundant context inflation), two handles; which fires is a property of the gate. (We earlier split these as two categories — *ctx_corpus_inflation* (2) and *ctx_depth_inflation* (2) — but they are one mechanism with two routes, merged 2026-06-26.)

Route A — corpus bloat (*corpus_count_ratio*). A deep, repeatedly-entered code path makes the *same* edges look novel once per call-context, so *naive_ctx* hoards far more (context-redundant) corpus entries than *naive*. The bloated corpus spreads the mutation budget thin — the fuzzer is busy re-saving the same control flow under fresh context hashes instead of mutating the bytes that flip the gate.

```
// libpng pngutil.c : png_read_IDAT_data()
} while (ret != Z_STREAM_END);           // deep inflate do/while loop
if (png_ptr->zstream.avail_in > 0 ||      // <-- blocker (libpng/4072):
    png_ptr->idat_size > 0)              // residual IDAT bytes left
    over
```

The branch needs residual compressed bytes after zlib returns `Z_STREAM_END`. *naive* flips it with ordinary havoc; but the deep inflate loop is re-entered 2750 times per loser seed (vs 136 per winner), and `naive_ctx` folds a context hash into each iteration’s edges, so every revisit looks like new coverage. It hoards a corpus 3.7× larger than *naive*’s and never spends the budget on the byte-level mutation that manufactures the extra-data condition.

Tool: `corpus_queue` (saved-corpus count, per trial)
Metric: `corpus_count_ratio` = median(saved entries, *naive_ctx*) / median(saved entries, *naive*)
Verify rule (deterministic arbiter): `winner = naive_ctx AND corpus_count_ratio ≥ 1.5`
Real example: `libpng/4072 (png_read_IDAT_data:4245)` — *naive_ctx* saves 16,761 entries vs *naive*’s 4,500: `corpus_count_ratio`= 3.73. `lcms/2149 (_cmsReadHeader:776, the 128-byte-header underflow gate)` shows the same sign at 1095 vs 636: `corpus_count_ratio`= 1.72.

Route B — depth fragmentation (`depth_lift`). Here the gate sits at the bottom of a tight, deeply-iterated path (a scan loop or a wide `switch`). *naive*’s flat per-edge map lets a single seed drive the path *deep* and converge on the exact operand; *naive_ctx* shatters that path’s edges across call-context slots, diluting the feedback so no seed reaches the gate depth the branch needs. The fingerprint is a per-seed *depth lift* on the resolving arm: the winner reaches the gate far deeper than the `ctx` loser ever does.

```
// libxml2 parser.c : xmlParseCharData()
do {                                     // tight char-data scan loop
    ...
    if (*in == 0xD) { ... continue; }    // CR handler keeps the loop
    alive
} while (IS_CHAR(*in));                 // <-- blocker (libxml2/6576:4610)
```

The gate needs a long char-data run with embedded CR/LF that keeps the `do-while` alive. *naive* assembles 1862-byte bodies that loop deep (per-seed depth 97.5); *naive_ctx*’s context-split edges dilute the scan-loop signal so its seeds stall shallow (22.2) and the loop falls through to `xmlParseCharDataComplex`, leaving the true side blocked.

Tool: `depth_reach` (`bench/tools/depth_reach.py`)
Metric: `depth_lift` = `winner_depth` / `loser_depth` (per-seed reach at the blocker line, *naive* vs *naive_ctx*); `winner_deeper`
Verify rule (deterministic arbiter): `depth_lift ≥ 2.0 AND winner_deeper == True`
Real example: `libxml2/6576 (xmlParseCharData:4610)` — `winner_depth` 97.5 vs `loser_depth` 22.2: `depth_lift`= 4.39. `harfbuzz/6124 (_hb_preprocess_text_vowel_constraints:409, the Tirhuta case 0x1148Bu/0x1148Du)` — *naive* converges on the exact codepoint (depth 2.0) while the `ctx` arm disperses across sibling Tirhuta cases (depth 0.1): `depth_lift`= 4.0.

2. The residual tail: shallow-literal dilution *inconclusive* ($n = 2$)

Two members — `libxml2/6605 (name[1]=='m'/'M')` and `harfbuzz/5802 (the paired SHIN+SIN-DOT codepoint)` — are *shallow* exact-literal equality gates. The mechanism story is plausible (*naive_ctx* splits the per-edge reward across context slots so the winning literal is never assembled), but it is *not discriminable* by any built corpus measurement. Because `ctx` is a non-local feedback whose footprint a shallow 1-hop gate’s corpus does not preserve, all four permitted handles were scored across the labeling rounds and *none* held: *corpus size* (`corpus_size_ratio`< 1.5), *call-depth* (`call_count_lift`~ 1, no loop to flood), *fixed-offset byte presence* (`signed.target_enrich` did not reach ≤ -0.5), and *offset-tolerant window distance* (`value.distance_reached`: no sustained

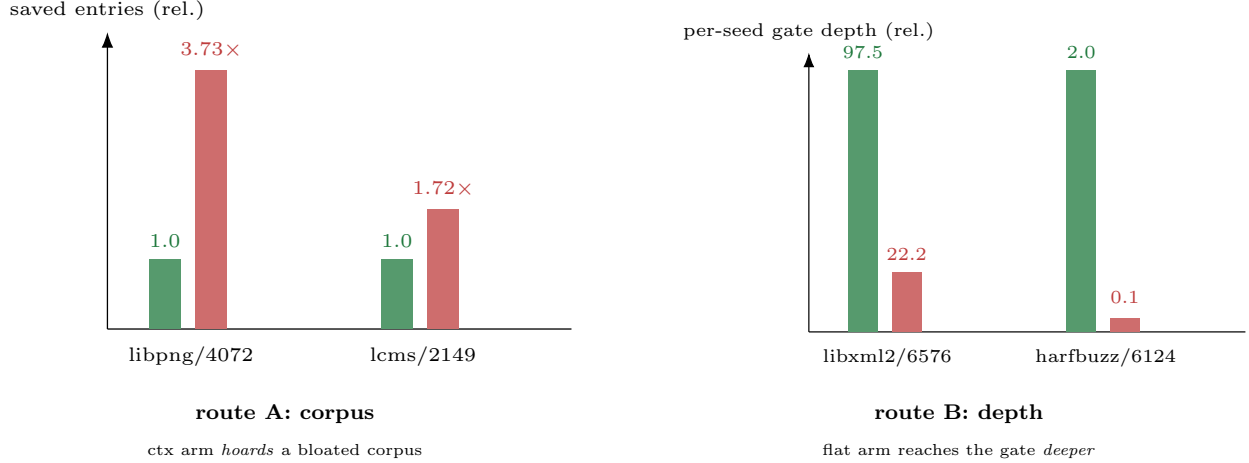


Figure 1: The two routes of `ctx_inflation`. **Left (route A):** corpus bloat — the context arm (*naive_ctx*, red) hoards a corpus $\geq 1.5\times$ larger than the flat arm (*naive*, green), because the same edges look novel under each call-context; bars are normalized to `naive`=1. **Right (route B):** depth fragmentation — the flat arm (green) drives the gate far deeper per seed than the context arm (red), which fragments the deep path across context slots; bars are absolute per-seed reach at the blocker line. In both, the arm with the *context* dimension is the one that blocks.

approach gap). The remaining built tools (`joint_necessity`, `token_count`) measure structural *assembly* for a different (`vpc-vs-cmplog`) subtype and do not fit a single-byte equality gate. Per guardrail G3, an honest *inconclusive* is the correct outcome here, not a forced label.

At a glance

Category	<i>n</i>	Example	How the context dimension backfires
<code>ctx_inflation</code>	4	libpng/4072 (A), libxml2/6576 (B)	route A deep re-entered path $\Rightarrow$ same edges look novel per context $\Rightarrow$ ctx arm hoards a bloated corpus (<code>corpus_count_ratio</code> ≥ 1.5); route B tight deep loop / wide switch $\Rightarrow$ ctx fragments the path $\Rightarrow$ flat arm reaches the gate deeper (<code>depth_lift</code> ≥ 2.0)
(<i>shallow-literal dilution</i>)	2	libxml2/6605	non-local feedback, no corpus footprint at a 1-hop gate $\Rightarrow$ honestly <i>inconclusive</i> (G3)
CTX-A validated	4		decisive pair <i>naive</i> $\succ$ <i>naive_ctx</i> throughout

All counts and metric values are from `bench/dataset.jsonl`, the per-branch analysis cards, and the `step5b_new_v3/ctx_coverage_LW` arbiter assignments; the verify rules are the deterministic arbiter rules (no LLM in the decision). `lcms/2149` and `libpng/4072` are corpus targets on server sB; `harfbuzz/6124` on s4; `libxml2/6576` on sB — each scored by its owning server.